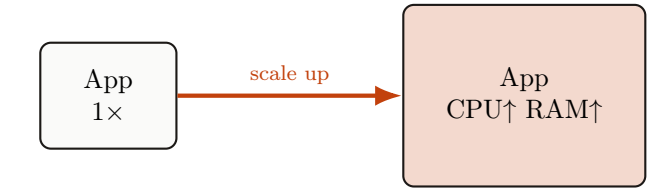
Scaling and Load Balancing

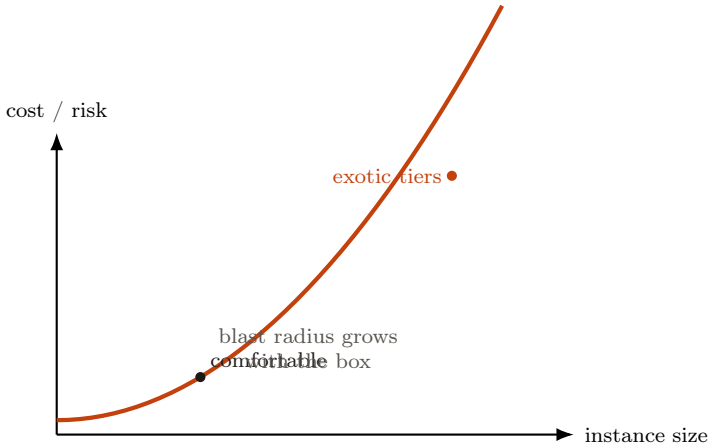
Analysis and Assessment of Design Quality

Scale up — one bigger box



More resources in one instance. No partition logic — hard ceiling remains.

Vertical — capacity vs cost (sketch)



When scale-up is enough (for now)

Hard to partition (single writer)

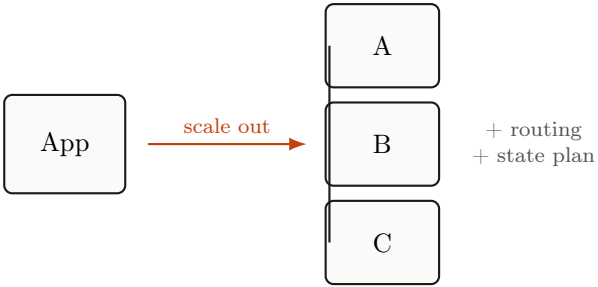
Low ops maturity for multi-node

Needs local memory / disk locality

Bridge while you design scale-out

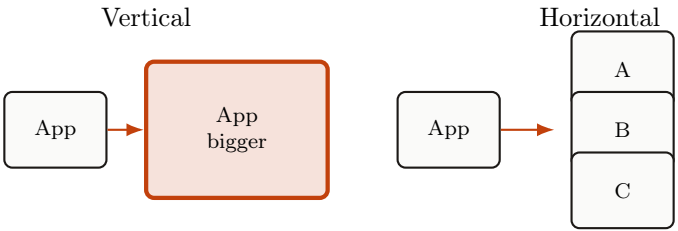
Review: next size tier? instance death RTO?

Scale out — more boxes



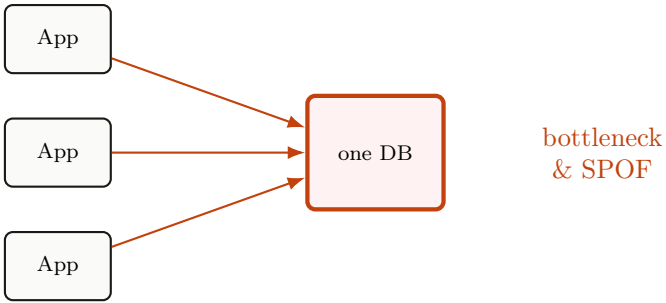
Capacity grows if work partitions. Compute without a data plan is incomplete.

Scale-up vs scale-out



Same goal (more capacity) — different failure modes and ops.

Scale-out needs a data story



Eight processes + one shared schema ≠ horizontal scale of the system.

Quiz — scaling choice

Quiz

Payment ledger needs serialisable balance updates; peak 3×. Best statement?

1. Horizontal scale of the process alone always solves it
2. Vertical may buy time; scale-out needs partition / concurrency design
3. LB algorithms remove the need for consistency
4. Stateless replicas + in-process cache are enough

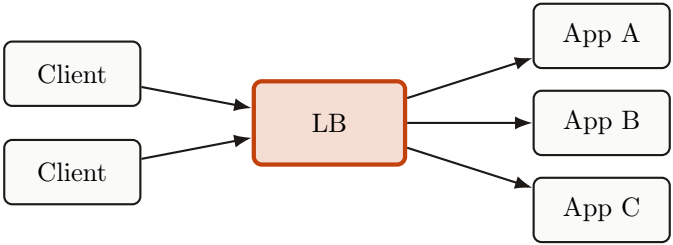
Pause — discuss with a neighbour or answer privately before the next slide.

Answer — scaling choice

Answer

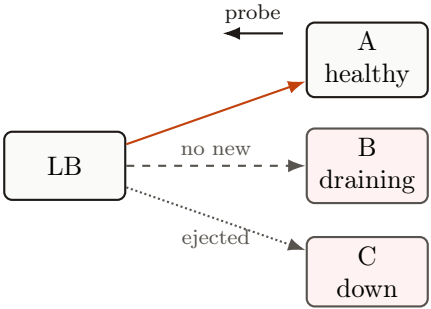
2. Serialisable updates do not magically partition; compute scale-out still needs a data story.

LB in front of N backends



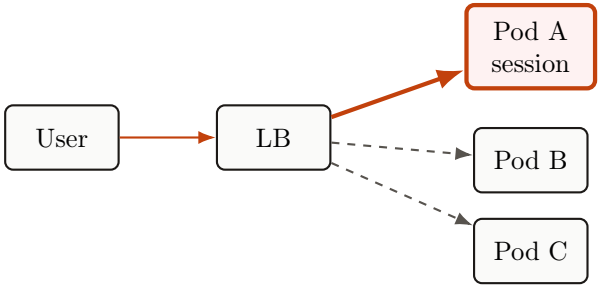
Hides pool membership; enables scale-out and rolling deploy (L4 vs L7).

Health check + drain



Active probes + passive ejection; drain before kill — else deploy = 5xx storm.

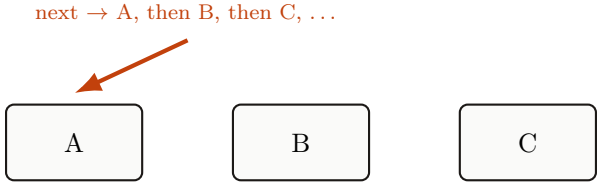
Sticky session pitfall



A dies ⇒
sessions gone
scale-in hurts

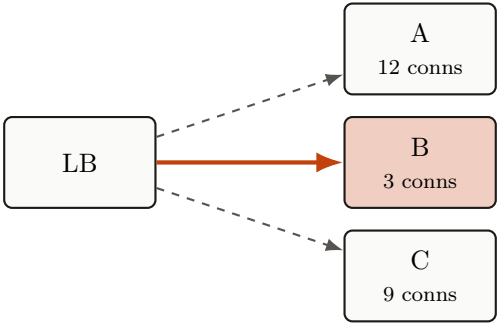
Affinity hides stateful debt — prefer external session store.

Round-robin pointer



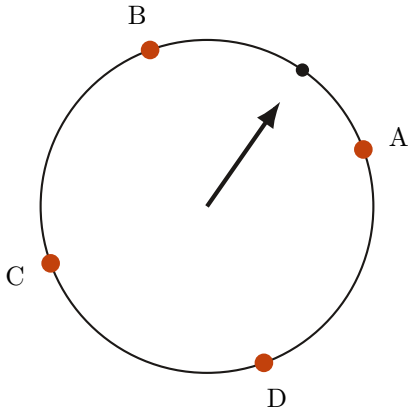
Fair only if request cost ≈ equal

Least connections (sketch)



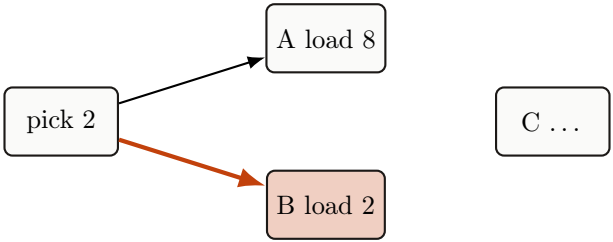
Prefers fewer open connections — still blind to CPU-heavy short calls.

Consistent hash ring



key hashes onto ring → next node clockwise
node join/leave remaps only nearby keys

Power of two choices (P2C)



send to the less loaded of the two

Simple default for many stateless HTTP pools.

Quiz — LB algorithm

Quiz

Identical backends, short HTTP, one node briefly slow (GC). Best default?

1. IP-hash stickiness onto the slow node
2. RR or P2C + health / passive ejection — not stickiness
3. Consistent hash on user id with no failover
4. Clients hard-code random IPs

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — LB algorithm

Answer

2. Even spread + eject the sick node beats sticky affinity to GC pauses.

Review snapshot

- ☐ vertical ceiling / SPOF?
- ☐ scale-out + data plan?
- ☐ LB HA + drain?
- ☐ algorithm fits load?
- ☐ timeouts / retries aligned?